C++ 類別與運算子多載

Week 2 教學講義

依課程第 11 章 Classes 與第 12 章 Operator Overloading 的主題安排；範例另行編寫。

目錄

1 類別、存取權限與建構子 1

2 成員函式與 const 2

3 比較運算子：== 與!= 3

4 複合指定運算子：+= 4

5 加法運算子：+ 4

6 索引運算子：[] 5

7 成員與類別外的運算子 6

本文把語法和短例子放在一起。每章最後的一句話對應同資料夾中〈學生練習〉的關卡。程式片段省略與該節無關的 `#include` 和 `main()`；完整可編譯的示範程式另見 `01_class_demo.cpp`、`02_operator_demo.cpp`。

1 類別、存取權限與建構子

C++ 的 `class` 可以把資料與操作資料的函式放在同一個型別中。以下的帳戶把餘額設為 `private`，因此類別外不能直接寫 `account.balance = -10`。類別外只能使用 `public` 中宣告的功能。

```
class BankAccount {
private:
    int balance;
public:
    BankAccount() : balance(0) {}
    bool deposit(int amount);
    int getBalance() const { return balance; }
};
```

`BankAccount()` 是建構子：建立物件時自動執行。它與類別同名，沒有回傳型別；`: balance(0)` 把新帳戶的餘額初始化為 0。類別定義末尾的分號不能省略。

deposit 在類別內宣告，也可在類別外定義。名稱前的 BankAccount:: 表示它是 BankAccount 的成員函式。

```
bool BankAccount::deposit(int amount) {
    if (amount <= 0) return false;
    balance += amount;
    return true;
}
```

例如新建帳戶後呼叫 deposit(50)，輸入的 50 是正數，因此餘額從 0 變成 50，函式回傳 true。接著呼叫 deposit(-3) 會回傳 false，餘額仍是 50。getBalance() 讓外面讀取結果，卻不開放直接改寫私有欄位。

```
BankAccount account;
cout << account.deposit(50) << ' ' << account.getBalance();
// 1 50
```

struct 和 class 都能有資料與成員函式。未標明存取權限時，struct 的成員預設為 public，class 的成員預設為 private。

接下來可以練習第 1 關

2 成員函式與 const

成員函式操作的是呼叫它的物件。若函式會改變物件，就不能讓 const 物件呼叫；若函式只讀資料，在參數列表後加上 const。

```
class Counter {
    int count;
public:
    Counter(int start) : count(start) {}
    void add(int amount) { count += amount; }
    int value() const { return count; }
};
```

add 的輸入是要增加的數量，執行後 count 會改變。value 只回傳目前數值，所以宣告為 const 成員函式。

```
Counter a(5);
a.add(2);               // a 的值變成 7
const Counter fixed(5);
cout << fixed.value();   // 印出 5
// fixed.add(2);         // 錯誤：不能修改 const 物件
```

把物件傳給只讀函式時，常用 const 型別 & 作參數。& 讓函式使用呼叫端原本的物件，不另複製一份；const 保證這個參數不能被函式修改。

```
void show(const Counter& counter) {
    cout << counter.value();
}
```

show 是類別外的一般函式，不屬於 Counter。它的 const 只修飾參數 counter，表示不能透過這個參數修改傳入的物件；一般函式不能在參數列表後加 const。

對照類別內的 int value() const：value 是成員函式，最後的 const 表示它不能修改呼叫它的物件。因此 show 可以用唯讀的 counter 呼叫 counter.value()。後面的比較與加法運算子也會用到這兩種 const 寫法。

接下來可以練習第 2 關

3 比較運算子：== 與!=

operator overloading（運算子多載）讓自訂型別指定既有運算子的行為。當 operator== 是成員函式時，a == b 可視為呼叫 a.operator==(b)：a 是左邊物件，b 是右邊參數。

```
class Box {
    int value;
public:
    Box(int initial) : value(initial) {}
    bool operator==(const Box& other) const {
        return value == other.value;
    }
};
```

輸入兩個 Box 時，函式比較它們的 value，輸出 bool。參數上的 const 保護右邊物件；函式最後的 const 保護左邊物件。若 a 與 b 的值都是 5，a == b 為真；若其中一個是 8，結果為假。

!= 可以重用 == 的結果，不必把比較條件再寫一次。

```
bool operator!=(const Box& other) const {
    return !(*this == other);
}
```

這裡的 this 是指向左邊物件的指標；*this 取得左邊物件本身。! 把 == 的真與假反過來。此函式也應放在 Box 的 public 區域。

接下來可以練習第 3 關

4 複合指定運算子：+=

a += b 應修改左邊的 a，右邊的 b 保持不變。以下 Counter 例子把右邊的數值加到左邊。

```
Counter& operator+=(const Counter& other) {
    count += other.count;
    return *this;
}
```

other 用 const Counter& 傳入，因此不需複製，也不能在函式內修改右邊。count += other.count 改變左邊物件的欄位。

回傳型別是 Counter&：return *this; 回傳左邊原本的物件，而不是新建物件。這讓 (a += b) += c 可以在第一次加完後，繼續修改同一個 a。

```
Counter a(2), b(3), c(4);
(a += b) += c;
cout << a.value();  // 9
cout << b.value();  // 3
```

若執行 a += a，左邊與右邊其實是同一個物件；此例會讓 a 的值加倍。寫自訂 += 時也要測這種情況。

接下來可以練習第 4 關

5 加法運算子：+

a + b 應產生一個新結果，並保留 a、b 的原值。與會修改左邊的 += 相比，+ 的回傳型別應是新物件的值。

```
Counter operator+(const Counter& other) const {
    Counter result(*this);
    result += other;
    return result;
}
```

result(*this) 先複製左邊物件到新的 result；result += other 只改這份新物件；最後把它的值回傳。因為左邊與右邊都沒有被修改，這個方法可宣告為 const。

```
Counter a(2), b(3);
Counter c = a + b;
cout << c.value();  // 5
cout << a.value();  // 2
```

不能回傳 Counter& 指向 result：result 是函式內的區域物件，函式結束後便不存在。return result; 回傳的是它的值，因此呼叫端仍可使用新結果。

接下來可以練習第 5 關

6 索引運算子：[]

operator[] 讓物件可以用 v[index] 存取內部資料。若想同時支援讀取和指定，必須區分「回傳值」與「回傳參考」。

```
int x = 2;
int copy = x;
copy = 9;       // x 仍是 2
int& alias = x;
alias = 9;      // x 變成 9
```

copy 是另一個整數；alias 是原本 x 的參考。若 v[0] 需要出現在指定運算的左邊，例如 v[0] = 9，它就必須回傳可修改欄位的參考。

```
int  operator[](int index) const; // const 物件：讀取值
int& operator[](int index);       // 一般物件：讀取或改寫
```

這兩個宣告可同時放在類別的 public 區域。一般物件會使用非 const 版本；const 物件只能使用最後有 const 的版本。

以下示範一個有兩個整數欄位的類別中，可修改版本的核心寫法。合法索引只有 0 與 1；其他索引必須明確處理，不能直接讀取無效位置。

```
int& operator[](int index) {
    if (index == 0) return first;
    if (index == 1) return second;
    std::exit(1);
}
```

std::exit(1) 需要 #include <cstdlib>；它會以非零狀態結束程式。const 版本也要檢查索引，但回傳整數值，而非可改寫的參考。

接下來可以練習第 6 關

7 成員與類別外的運算子

成員運算子由左邊的物件呼叫。因此 `box + 3` 可寫成 `Box` 的成員函式；`3 + box` 的左邊是 `int`，不能呼叫 `Box` 的成員函式。

先定義 `Box` 加整數的意思是「內部數值增加該整數，產生新的 Box」。在第 3 節的 `Box` 類別中新增以下方法：

```
Box operator+(int amount) const {
    return Box(value + amount);
}
```

要支援反過來的順序，可在類別外寫一個有兩個參數的運算子。它呼叫上面的公開版本，不需直接碰 `Box` 的私有欄位。

```
Box operator+(int amount, const Box& box) {
    return box + amount;
}
```

```
Box box(5);
Box a = box + 3;  // 值為 8
Box b = 3 + box;  // 值也為 8
```

本章用到的運算子可按是否改變左邊物件來區分：

運算	左邊物件會改變嗎？	典型回傳型別
`a == b`	不會	`bool`
`a += b`	會	`T&`
`a + b`	不會	`T`
`v[index] = n`	會	`int&`

接下來可以練習第 7 關